

FRED TALKS

26th August 2026

CONTENTS

How Claude Code navigates large codebases

CLAUDE · 2694 WORDS

It's-a Me, Agentic AI

HAN, NOT SOLO · 2362 WORDS

Trigger scheduled tasks from Gmail, Slack, and GitHub events

CODEx CHANGELOG · 146 WORDS

How Claude Code navigates large codebases

CLAUDE · 18 MAY 2026 · [SOURCE](#)

Claude Code is running in production across multi-million-line monorepos, decades-old legacy systems, distributed architectures spanning dozens of repositories, and at organizations with thousands of developers. These environments present challenges that smaller, simpler codebases don't, whether that's build commands that differ across every subdirectory or legacy code spread across folders with no shared root.

This article covers the patterns we've observed that have led to successful adoption of Claude Code at scale. We use "large codebase" to refer to a wide range of deployments: monorepos with millions of lines, legacy systems built over decades, dozens of microservices across separate repositories, or any combination of the above. That also includes codebases running on languages that teams don't always associate with AI coding tools, such as C, C++, C#, Java, PHP. (Claude Code performs better than most teams expect it to in those cases, particularly as of recent model releases.) While every large codebase deployment is shaped by its specific version control, team structure, and accumulated conventions, the patterns here generalize across them and are a good starting point for teams considering adopting Claude Code.

Claude Code navigates a codebase the way a software engineer would: it traverses the file system, reads files, uses `grep` to find exactly what it needs, and follows references across the codebase. It operates locally on the developer's machine and doesn't require a codebase index to be built, maintained, or uploaded to a server.

RAG-powered AI coding tools work by embedding the entire codebase and retrieving relevant chunks at query time. At large scale, those systems can fail because embedding pipelines can't keep up with active engineering teams. By the time a developer queries the index, it reflects the codebase as it previously existed weeks, days, or even hours before. Retrieval then returns a function the team renamed two weeks ago, or references a module that was deleted in the last sprint, with no indication that either is out of date.

Agentic search avoids those failure modes. There's no embedding pipeline or centralized index to maintain as thousands of engineers commit new code. Each developer's instance works from the live codebase.

But the approach has a tradeoff: it works best when Claude has enough starting context to know where to look. This means the quality of Claude's navigation is shaped by how well the codebase is set up, layering context with CLAUDE.md files and skills. If you ask it to find all instances of a vague pattern across a billion-line codebase, you'll hit context-window limits before the work begins. Teams that invest in codebase setup see better results.

The harness matters as much as the model

One of the most common misconceptions about Claude Code is that its capabilities are solely defined by the model used. Teams focus on a model's benchmarks and how it performs on test tasks. In practice, the ecosystem built around the model—the harness—determines how Claude Code performs more than the model alone.

The harness is built from five extension points—CLAUDE.md files, hooks, skills, plugins, and MCP servers—each serving a different function. The order in which teams build them matters, as each layer builds on what came before. Two additional capabilities, LSP integrations and subagents, round out the setup. Below, we explain what each of these components and capabilities do:

CLAUDE.md files come first. These are context files that Claude reads automatically at the start of every session: root file for the big picture, subdirectory files for local conventions. They give Claude the codebase knowledge it needs to do anything well. Because they load in every session regardless of the task, keeping them focused on what applies broadly will prevent them from becoming a drag on performance.

Hooks make the setup self-improving. Most teams think of hooks as scripts that prevent Claude from doing something wrong, but their more valuable use is continuous improvement. A stop hook can reflect on what happened during a session and propose CLAUDE.md updates while the context is fresh. A start hook can load team-specific context dynamically so every developer gets the right setup for their module without manual configuration. For automated checks like linting and formatting, hooks enforce the rules deterministically and produce more consistent results than relying on Claude to remember an instruction.

Skills keep the right expertise available on-demand without bloating every session. In a large codebase with dozens of task types, not all expertise needs to be present in every session. Skills solve this through progressive disclosure, offloading specialized workflows and domain knowledge that would otherwise compete for context space and loading them only when the task calls for it. For example, a security review skill loads when Claude is assessing code for vulnerabilities, while a document processing skill loads when a code change is made and documentation needs to be updated.

Skills can also be scoped to specific paths so they only activate in the relevant part of the codebase. A team that owns a payments service can bind their deployment skill to that directory, so it never auto-loads when someone is working elsewhere in the monorepo.

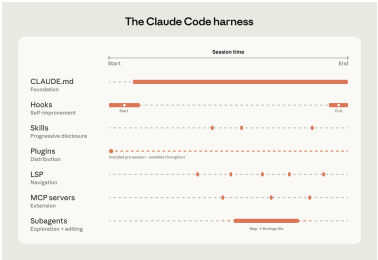
Plugins distribute what works. One challenge with large codebases is that *good* setups can stay tribal. A plugin bundles skills, hooks, and MCP configurations into a single installable package, so when a new engineer installs that plugin on day one, they will immediately have the same context and capabilities as those who have been using Claude already. Plugin updates can be distributed across the organization through managed marketplaces.

For example, a large retail organization we work with built a skill connecting Claude to their internal analytics platform so that business analysts could pull performance data without leaving their workflow. They distributed it as a plugin before the broad rollout to the business.

Language server protocol (LSP) integrations give Claude the same navigation a developer has in their IDE. Most large-codebase IDEs already have an LSP running, powering "go to definition" and "find all references." Surfacing this to Claude gives it symbol-level precision: it can follow a function call to its definition, trace references across files, and distinguish between identically named functions in different languages. Without it, Claude pattern-matches on text and can land on the wrong symbol. One enterprise software company we worked with deployed LSP integrations org-wide before their Claude Code rollout, specifically to make C and C++ navigation reliable at scale. For multi-language codebases, this is one of the highest-value investments.

MCP servers extend everything. MCP servers are how Claude connects to internal tools, data sources, and APIs that it can't otherwise reach. The most sophisticated teams built MCP servers exposing structured search as a tool Claude can call directly. Others connect Claude to internal documentation, ticketing systems, or analytics platforms.

Subagents split exploration from editing. A subagent is an isolated Claude instance with its own context window that takes a task, does the work, and returns only the final result to the parent. Once the harness is in place, some teams spin up a read-only subagent to map a subsystem and write findings to a file, then have the main agent edit with the full picture.



Claude Code’s extension layer at a glance.

The table below summarizes what each component does, when it loads, and the most common mistakes we see with each:

Component	What it is	When it loads	Best for	Common confusion
CLAUDE.md	Context file Claude reads automatically	Every session	Project-specific conventions, codebase knowledge	Using it for reusable expertise that belongs in a skill
Hooks	Scripts that run at key moments	Triggered by events	Automating consistent behavior, capturing session learnings	Using prompts for things that should run automatically
Skills	Packaged instructions for specific task types	On demand, when relevant	Reusable expertise across sessions and projects	Loading everything into CLAUDE.md instead
Plugins	Bundled skills, hooks, MCP configs	Always available once configured	Distributing a working setup across the org	Letting good setups stay tribal
Language server protocol (LSP)*	Real-time code intelligence via language specific servers	Always available once configured	Symbol-level navigation and automatic error detection in typed languages	Assuming that it's automatic
MCP servers	Connections to external tools and data	Always available once configured	Giving Claude access to internal tools it can't otherwise reach	Building MCP connections before the basics are working

Subagents*	Separate Claude instances for specific tasks	When invoked	Splitting exploration from editing, parallel work	Running exploration and editing in the same session

*LSP is accessed through the plugin layer. Subagents are a delegation capability rather than a configured extension point.

Three configuration patterns from successful deployments

How you configure Claude Code for a large codebase depends heavily on how that codebase is structured. Still, three patterns appeared consistently across the deployments we observed.

Making the codebase navigable at scale

Claude's ability to help in a large codebase is bounded by its ability to find the right context. Too much context loaded into every session degrades performance, while too little context leaves Claude to navigate blind. The most effective deployments invest upfront in making the codebase legible to Claude. A few patterns appear consistently:

- **Keeping CLAUDE.md files lean and layered.** Claude loads them additively as it moves through the codebase: root file for the big picture, subdirectory files for local conventions. The root file should be pointers and critical gotchas only; everything else drifts into noise.
- **Initializing in subdirectories, not at the repo root.** Claude works best when it's scoped to the part of the codebase that's actually relevant to the task. In monorepos, this can feel counterintuitive because tooling often assumes root access, but Claude automatically walks up the directory tree and loads every CLAUDE.md file it finds along the way, so root-level context is never lost.
- **Scoping test and lint commands per subdirectory.** Running the full suite when Claude changed one service causes timeouts and wastes context on irrelevant output. CLAUDE.md files at the subdirectory level should specify the commands that apply to that part of the codebase. This works well for service-oriented codebases where each directory has its own test and build commands. In compiled-language monorepos with deep cross-directory dependencies, per-subdirectory scoping is harder to achieve and may require project-specific build configurations.

- **Using `.ignore` files to exclude generated files, build artifacts, and third-party code.** Committing `permissions.deny` rules in `.claude/settings.json` means the exclusions are version-controlled, so every developer on the team gets the same noise reduction without configuring it themselves. In some codebases, generated files are themselves the subject of development work. Developers who work on code generators can override project-level exclusions in their local settings without affecting the rest of the team.
- **Building codebase maps when the directory structure doesn't do the work.** For organizations where code isn't consolidated in a conventional directory structure, a lightweight markdown file at the repo root listing each top-level folder with a one-line description of what lives there gives Claude a table of contents it can scan before opening files. For codebases with hundreds of top-level folders, this works best as a layered approach: the root file describes only the highest-level structure, and subdirectory `CLAUDE.md` files provide the next level of detail, loading on demand as Claude moves through the tree. For simpler cases, `@`-mentioning the specific files or directories Claude should reference can do the same job.
- **Running LSP servers so Claude searches by symbol, not by string.** Grep for a common function name in a large codebase returns thousands of matches and Claude burns context opening files to figure out which matters. LSP returns only the references that point to the same symbol, so the filtering happens before Claude reads anything. Setting this up requires installing a [code intelligence plugin](#) for your language and the corresponding language server binary; the Claude Code documentation covers the available plugins and troubleshooting.

One caveat: there are edge cases where even the hierarchical `CLAUDE.md` approach breaks down, for example codebases with hundreds of thousands of folders and millions of files, or legacy systems on non-git version control. We will address their challenges in future installments of this series.

Actively maintaining `CLAUDE.md` files as model intelligence evolves

As models evolve, instructions written for your current model can work against a future one. `CLAUDE.md` files that guided Claude through patterns it used to struggle with may either become unnecessary or actively constraining when the next model ships. For example, a `CLAUDE.md` rule that tells Claude to break every refactor into single-file changes may have helped an earlier model stay on track but would prevent a newer one from making coordinated cross-file edits it handles well.

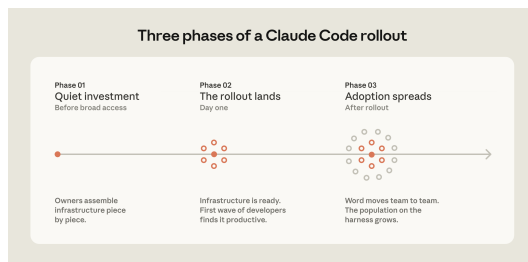
Skills and hooks built to compensate for specific model limitations, whether in the model's reasoning or in Claude Code's own tooling, become overhead once those limitations no longer exist. A hook that intercepted file writes to enforce p4 edit in a Perforce codebase, for example, became redundant once Claude Code added native Perforce mode.

Teams should expect to do a meaningful configuration review every three to six months, but it's also worth doing one whenever performance feels like it's plateaued after major model releases.

Assigning ownership for Claude Code management and adoption

Technical configuration alone doesn't drive adoption. The organizations that got it right invested in the organizational layer, too.

The rollouts that spread fastest had a dedicated infrastructure investment before broad access. A small team, sometimes even just one person, wired up the tooling so Claude already fit developer workflows when they first touched it. At one company, a couple of engineers built a suite of plugins and MCPs that were available on day one. At another, an entire team focused on managing AI coding tools had the infrastructure in place before the rollout began. In both cases, developers' first experience was productive rather than frustrating, and adoption spread from there.



The teams doing this work today tend to sit under developer experience or developer productivity, which is typically the function responsible for onboarding new engineers and building developer tooling. An emerging role in several organizations is an agent manager: a hybrid PM/engineer function dedicated to managing the Claude Code ecosystem. For organizations without a dedicated team, the minimum viable version is a DRI: one person with ownership over the Claude Code configuration, the authority to make calls on settings, permissions policy, the plugin marketplace, and CLAUDE.md conventions, and the responsibility to keep them current.

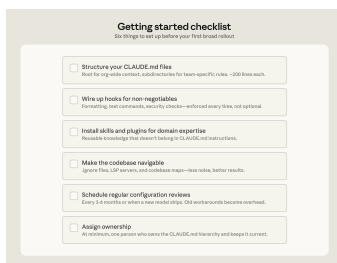
Bottoms-up adoption generates enthusiasm but can fragment without someone to centralize what works. You need to have an individual or a team assemble and evangelize the right Claude Code conventions (such as a standardized CLAUDE.md hierarchy or a curated set of skills and plugins). Without that work, knowledge will stay tribal and adoption will plateau.

In large organizations, especially those in regulated industries, governance questions come up early, such as: who controls which skills and plugins are available, how do you prevent thousands of engineers from independently rebuilding the same thing, how do you make sure AI-generated code goes through the same review process as human-generated code? To address these early on, we suggest starting with a defined set of approved skills, required code review processes, and limited initial access, and expand as confidence builds.

We've observed the smoothest deployments at organizations that establish cross-functional working groups early by bringing together engineering, information security, and governance representatives to define requirements together and build a rollout roadmap.

Applying these patterns to your organization

Claude Code is designed around conventional software engineering environments where engineers are the primary codebase contributors, the repo uses Git, and code follows standard directory structures. Most large codebases fit this mold, but non-traditional setups such as game engines with large binary assets, environments with unconventional version control, or non-engineers contributing to the codebase require additional configuration work. Our guidance assumes a conventional setup and the patterns we've described have worked across many of our customers. Any remaining complexity requires judgment specific to your codebase, tooling, and organization. That's where Anthropic's Applied AI team works directly with engineering teams to translate these patterns into your organization's specific requirements.



Getting started checklist
(All steps are not required for all organizations)

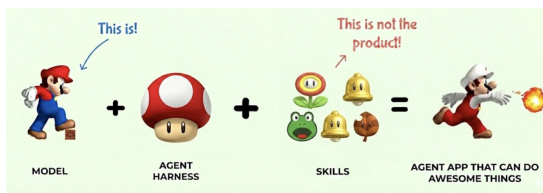
- ☐ **Structure your CLAUDE.md files**
Start for org-wide context, add structure for team-specific rules. >200 lines each.
- ☐ **Wipe up hooks for non-registrars**
Formatting, test commands, security checks—optional every time, not optional.
- ☐ **Install skills and plugins for domain expertise**
Reusable knowledge that doesn't belong in CLAUDE.md instructions.
- ☐ **Make the codebase navigable**
Ignore files, LSP servers, and codebase maps—more index, better results.
- ☐ **Schedule regular configuration reviews**
Every 3-6 months or when a new model ships. Old instructions become outdated.
- ☐ **Assign ownership**
At least one, one person who owns the CLAUDE.md hierarchy and keeps it current.

Acknowledgements: Special thanks to Alon Krifcher, Charmaine Lee, Chris Concannon, Harsh Patel, Henrique Savelli, Jason Schwartz, Jonah Dueck and Kirby Kohlmorgen from Anthropic's Applied AI team for sharing their experience deploying Claude Code at scale, and to Amit Navindgi at Zoex for providing feedback on this article.

It's-a Me, Agentic AI

HAN, NOT SOLO · 01 MAR 2026 · [SOURCE](#)

Agentic AI is a fairly recent development that combines reasoning ([OpenAI, 2024](#)) tool use ([Schick et al, 2023](#)) in the same AI model. But an agentic AI system is not just the model, but also the harness, the environments, the tools, rewards, evaluations, and all of the infrastructure to support it. In this post, let's use the top grossing franchise, Mario, to tell this story for understanding how agentic AI models are developed, how agent harnesses work, and how reinforcement learning ties everything together. If you survived World 8-4 as a kid, you already have the intuition for building agentic AI systems.



alt text

Small Mario is the **base pretrained model**. He's just come out of pretraining on a massive corpus of platform game physics. He can walk. He can jump. He can move left and right. These are his base capabilities, the raw knowledge compressed from the training data.

But Small Mario is fragile. One hit from a Goomba and he's dead. He can't break bricks. He can't take damage. He has potential, but he's not yet useful for anything beyond the most trivial tasks.

This is your base LLM fresh off pretraining. It has absorbed enormous amounts of knowledge, it can do next-token prediction, and it can sort-of follow instructions. But ask it to do anything real, reliably, in production, and it falls apart on the first obstacle. One Goomba and it's game over.

The Super Mushroom: Model Harness

Then Mario finds the **Super Mushroom**. He doubles in size. He can now break bricks. He can take a hit without dying. He goes from fragile to capable.

The Super Mushroom is the **model harness**, the entire infrastructure and post-training pipeline that transforms a base model into something production-ready. This includes:

- **Instruction tuning** (RLHF, DPO, etc.) to make the model actually follow directions
- **System prompts** that define personality and constraints
- **Safety guardrails** so it doesn't run off cliffs
- **Memory and context management** so it remembers where it's been
- **Tool-use training** so it knows power-ups exist and how to grab them

Without the Super Mushroom, Mario is a liability. With it, he's a platform for greatness. Similarly, without the model harness, a base LLM is a research artifact. With it, it's a product.

The Super Mushroom doesn't change WHO Mario is. It changes what he can SURVIVE. The model harness doesn't change the model's core knowledge. It changes what the model can handle in production.

Now here's where it gets interesting. Super Mario can pick up **power-ups** that give him entirely new capabilities:

Power-Up	Mario Ability	Agent Equivalent
Fire Flower	Throw fireballs to defeat enemies at range	Code execution - reach out and run code to solve problems the model can't solve with text alone
Frog Suit	Swim through water levels with ease	Web search - navigate and retrieve information from an environment the model wasn't trained on
Raccoon Leaf	Fly and reach otherwise inaccessible areas	File system access - explore and modify the codebase, reach files the model couldn't otherwise touch
Hammer Suit	Throw hammers, defeat almost anything	Shell/terminal access - the most powerful tool, can interact with the full system
Star	Temporary invincibility, blow through everything	Extended thinking/reasoning mode - brute force through complex problems at higher compute cost
Cape Feather	Sustained flight and spin attack	MCP servers - sustained, extensible access to external services and APIs

Each power-up doesn't replace Mario's core abilities. Mario still walks and jumps. The power-ups **extend** what he can do. A Fire Flower Mario can still jump on Goombas, but now he can also shoot fireballs at Piranha Plants hiding in pipes.

This is exactly how agent tools work. The LLM still does what LLMs do: reasoning, language understanding, and planning. Tools extend the model's reach into environments it can't operate in alone. An LLM can't execute Python by itself, just like Mario can't throw fireballs without a Fire Flower. But give it the right tool, and suddenly the problem space opens up.

And critically, **Mario has to learn WHEN to use each power-up**. Frog Suit is amazing in water levels, useless on land. Fire Flower is great against Goombas, pointless against Thwomps. The model needs to learn tool selection, knowing which tool to reach for in which context. This is one of the hardest parts of building agentic systems.

One power-up deserves special attention: the **Star**. When Mario grabs a Star, he becomes invincible. He plows through Goombas, Koopa Troopas, Piranha Plants, everything in his path just disintegrates. Nothing can stop him. This is not dissimilar to having an engineering manager who's really good at clearing organizational blockers for their engineers. The Goombas and Piranha Plants of bureaucracy, cross-team dependencies, access requests, and priority conflicts just melt away. Star power is temporary and expensive, but when you need to blast through a critical path, nothing else comes close.

The Mushroom Kingdom: Environments and Tasks

Now let's talk about the world Mario operates in. Every level in the Mushroom Kingdom is an **environment**, and every environment is composed of the same building blocks:

Level Element	Environment Equivalent
Bricks	Structured data, APIs, databases that can be broken open to reveal resources
? Blocks	Unknown information sources, sometimes containing exactly what you need
Pipes	Entry points to sub-tasks, function calls, or deeper exploration
Coins	Incremental progress signals, intermediate rewards
Goombas	Common obstacles, predictable errors, edge cases
Koopa Troopas	Recyclable obstacles, errors whose solutions can be reused as tools
Piranha Plants	Traps hiding in seemingly safe passages, hallucination triggers
Vines	Hidden paths to bonus areas, unexpected shortcuts

Clouds / Platforms

Abstractions and scaffolding that support traversal

Pits

Catastrophic failures, unrecoverable errors

Every level remixes these elements differently. World 1-1 is simple, a few Goombas, some bricks, a clear path to the flag. World 8-4 is a maze of pipes, hidden paths, and a boss fight with Bowser. Same building blocks, radically different difficulty.

Throughout each level, Mario interacts with the environment as **tool use**. He enters pipes to warp from one place to another, the equivalent of an API call that transports you to an entirely different context. He bumps ? Blocks from below to discover power-ups, new agent skills materializing from the environment when you know where to look. He breaks bricks to clear paths or reveal hidden rewards, structured data yielding its value when you apply force in the right direction. He stomps on a Koopa shell and kicks it forward, turning an obstacle into a projectile that clears a line of Goombas, repurposing error outputs as inputs to solve downstream problems. The environment isn't just a backdrop. It's a toolbox.

Having an environment is not enough. We need to define **what we want to achieve** from playing the game. These are the **tasks**.

Mario can play the same level with completely different objectives. Complete the level. Complete the level in the shortest amount of time. Get the highest score. Collect the most coins. Accumulate the most 1-up lives. Find all the hidden rewards. Stomp on every last Goomba. Each objective produces a fundamentally different play style from the same environment.

This is exactly the task definition problem in agentic AI. The same model operating in the same environment can be pointed at wildly different goals. “Summarize this codebase” and “refactor this codebase” use the same files, the same tools, the same context, but they require entirely different strategies. The task is what transforms an environment from a sandbox into a mission.

The Flagpole: Evaluations and Reward Modeling

At the end of every level, there's a **flagpole**. Mario jumps on it, pulls down the flag, and receives a reward. The higher he grabs the flag, the bigger the reward. Some levels end with a boss fight against Bowser, where the reward is freeing a Toad (or eventually, the Princess).

This is the **reward signal** in reinforcement learning. But here's the critical question: how do we actually **measure** how well the game was played? This is **evaluation**, or **reward modeling**, and it is where the machine learning engineering discipline really shines. (or research engineer, applied AI engineer.)

The evaluation could be the raw score. It could be the number of 1-ups gained. Coins collected. Goombas stomped. Time remaining. Different paths discovered. Most frequently, it is a **combination** of some or all of the above, weighted and balanced against each other. Do we reward Mario more for speed or for thoroughness? For survival or for aggression? For finding secrets or for staying on the critical path?

Evaluation Metric	Mario Measure	Agent Measure
Speed	Time remaining on the clock	Task completion latency
Score	Points accumulated	Overall output quality
Collection	Coins gathered	Information retrieved, resources used efficiently
Completeness	Hidden blocks found, secrets discovered	Edge cases handled, comprehensive coverage
Efficiency	Enemies defeated per life	Correct tool invocations per task
Exploration	Different paths taken	Novel approaches discovered

Designing these rewards is a strict machine learning engineering discipline. A poorly shaped reward function produces an agent that technically completes tasks but in degenerate ways, like a Mario speedrunner who clips through walls. Impressive, but not what we actually wanted. Reward hacking is the Goodhart’s Law of agentic AI: when a measure becomes a target, it ceases to be a good measure.

Here’s where the full picture comes together. How does Mario learn to complete levels?
Reinforcement learning.

Mario starts each level knowing nothing about its specific layout. He has to:

1. **Observe** the current state, what’s on screen, where the enemies are, what power-ups are available
2. **Decide** on an action based on his policy, jump, run, shoot, or wait
3. **Act** and receive feedback from the environment
4. **Update** his policy based on the outcome

This is the MDP (Markov Decision Process) loop. The same loop described in Agents Are Workflows. The same loop that every agentic AI system runs:

The value of Mario's current state equals the expected immediate reward plus the discounted value of the next state. Should Mario jump NOW to get the coin, or wait and avoid the Goomba? The optimal policy balances immediate rewards against future outcomes.

Through repeated play (training episodes), Mario learns:

- Which obstacles can be jumped on vs. avoided
- When to use power-ups vs. save them
- Which pipes lead to shortcuts vs. dead ends
- How to handle boss fights

An agentic AI model goes through the same process. Through reinforcement learning (PPO, DPO, GRPO, or whatever the latest acronym is), the model learns:

- Which tools to invoke for which subtasks
- When to think longer vs. act immediately
- Which approaches work for which problem types
- How to decompose complex tasks into manageable steps

So who's actually making all of this work? Mario doesn't train himself.

To teach agent Mario to be really good at the game, we employ an **Machine Learning Engineer (MLE)**. The MLE is the game designer and coach rolled into one. They construct mock environments, set up the harnesses and tools, define the tasks that Mario needs to achieve, and most importantly, **design the reward function**. The MLE decides what "good" looks like. Do we reward Mario for speed? Thoroughness? Both? How much? This reward design is the single highest leverage decision in the entire pipeline. Get it right and Mario learns to play beautifully. Get it wrong and Mario learns to exploit glitches.

Once the MLE has designed the training setup, the **Machine Learning Systems Engineers (MLSys)** take over. They are the ones who actually run the show at scale. They set up the environment and agent Mario across hundreds to hundreds of thousands of environments, tasks and iterations. They manage the compute, the distributed training runs, the data pipelines. They collect the reasoning traces, the sequences of observations, actions, and outcomes from every single episode Mario plays. And from these traces, they run the reinforcement learning algorithms that allow agent Mario to learn from experience.

This is the unsexy but critical part. An MLE can design the most elegant reward function in the world, but without MLSys engineers standing up the infrastructure to run millions of training episodes and collect the resulting data, Mario never gets past World 1-1.

Worlds: Agent Frameworks and Domains

The Mushroom Kingdom isn’t one level. It’s organized into **Worlds**, each with a distinct theme and set of challenges:

World	Theme	Agent Domain
World 1	Grassland, basics	Simple text tasks, Q&A, summarization
World 2	Desert	Data analysis, structured environments
World 3	Water	Web and API navigation
World 4	Giant Land	Large-scale refactoring, migrations
World 5	Sky	Architecture and system design
World 6	Ice	Debugging in fragile or legacy environments
World 7	Pipe Maze	Complex multi-step workflows with dependencies
World 8	Bowser’s Castle	Full autonomous task completion with adversarial conditions

Agent frameworks are the **World Maps** - they organize how the agent navigates between levels (subtasks) and worlds (domains). Some frameworks are rigid, you follow the path laid out. Others let you pick your route. The best ones (like how Super Mario Bros. 3 introduced the overworld map) let the agent make strategic choices about which level to tackle next.

But remember: the model is the product. The World Map (framework) is useful scaffolding, but Mario does the actual playing. As models get stronger, they need less scaffolding. Today’s World 8 is tomorrow’s World 1.

Conclusion

The entire agentic AI stack maps to Mario with surprising fidelity:

Mario	Agentic AI
Small Mario	Base pretrained model
Super Mushroom	Model harness and post-training
Power-ups	Agent tools and skills
Star Power	Engineering manager clearing blockers

Levels	Task environments
Pipes, bricks, shells	Tool use within the environment
Objectives (speed, score, coins)	Task definitions
Flagpole / Boss	Reward signal and evaluation
Reward design	Reward modeling (ML engineering discipline)
Learning to play	Reinforcement learning
MLE as game designer	ML engineer designing tasks and rewards
MLSys running millions of episodes	ML infrastructure for training at scale
World Map	Agent framework
Multiplayer	Multi-agent systems

The next time someone asks you to explain agentic AI, ask them: have you played Mario?

If they can understand that Small Mario needs a mushroom to survive, power-ups to be effective, and practice to master each level, they can understand agentic AI development.

Now go save Princess Peach.

References

```
@article{
  leehanchung,
  author = {Lee, Hanchung},
  title = {It's-a Me, Agentic AI},
  year = {2026},
  month = {02},
  day = {18},
  howpublished = {\url{https://leehanchung.github.io}},
  url = {https://leehanchung.github.io/blogs/2026/02/18/mario-agent}
}
```

Trigger scheduled tasks from Gmail, Slack, and GitHub events

CODEx CHANGELOG · 25 AUG 2026 · [SOURCE](#)

Trigger scheduled tasks from Gmail, Slack, and GitHub events ChatGPT scheduled tasks can now run when supported events occur in Gmail, Slack, or GitHub. Filter Gmail messages by sender or subject, watch selected Slack channels, or respond to pull request activity such as reviews, comments, commit updates, and merges. Event-triggered tasks are available in ChatGPT on the web and mobile for eligible plans. Connect the relevant app and approve its requested access before creating a task. The ChatGPT Slack app must be a member of each watched channel, and the connected GitHub app must have access to each watched repository. An event-triggered task can't also use a time-based schedule. When matching events arrive close together, ChatGPT may combine them in one run. Open **Scheduled** to review pending events or choose **Run now**. [Learn how to trigger scheduled tasks from app events](/codex/automations?surface=web#web-trigger-tasks-from-app-events).